4190.428 Introduction to Machine Learning

Perceptron & Neural Networks

Jaesik Park
CSE & IPAI, SNU
jaesik.park@snu.ac.kr

Perceptron

- The coolest algorithm name: **The Perceptron**
  - It was developed by Rosenblatt in 1962.
  - At the time, it was not interpreted as attempting to optimize any particular criteria.
  - It was presented directly as an algorithm.
  - There has, since, been a huge amount of study and analysis of its convergence properties and other aspects of its behavior.

Algorithm

- Recall that we have a training Dataset $\mathscr{D}_n$ with $\mathbf{x} \in \mathbb{R}^d$, and $\mathbf{y} \in \{-1, +1\}$.
- The perceptron algorithm trains a binary classifier $h(\mathbf{x}; \theta, \theta_0)$ using the following algorithm to find θ and θ_0 using τ iterative steps:

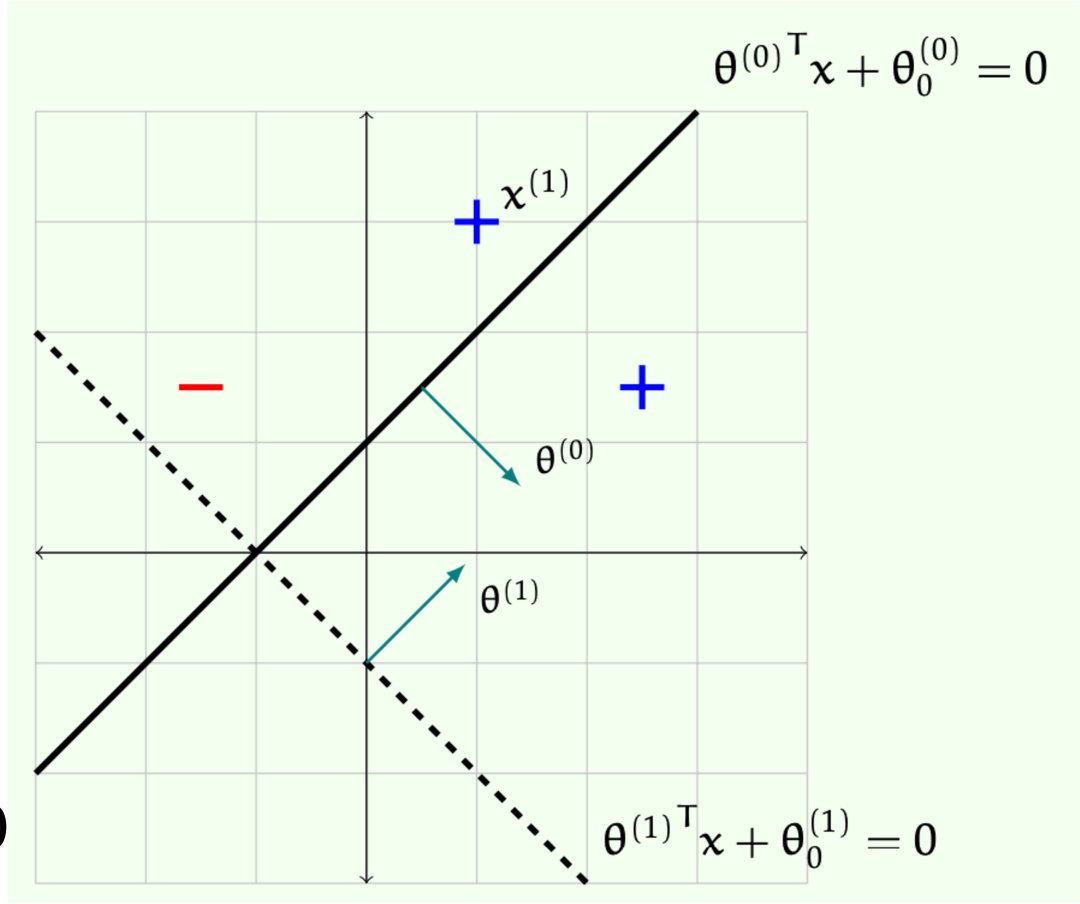
Algorithm 3: Perceptron$(\tau, \mathcal{D}_n)$

```
θ = [0  0  ···  0]^⊤
θ_0 = 0
for t = 1 to τ do
    for i = 1 to n do
        if y^(i)(θ^⊤ x^(i) + θ_0) ≤ 0 then
            θ = θ + y^(i) x^(i)
            θ_0 = θ_0 + y^(i)
return θ, θ_0
```

- **Study question**: What is true about ϵ_n if that happens?

Algorithm

- Example:
  - Let h be the linear classifier defined by $\boldsymbol{\theta}^{(\mathbf{0})} = \begin{bmatrix} 1 \\ -1 \end{bmatrix}, \ \theta_0^{(0)} = 1$
  - The diagram below shows several points classified by h.
  - But h misclassifies point $\mathbf{x}^{(\mathbf{1})} = \begin{bmatrix} 1 \\ 3 \end{bmatrix}, \ y^{(1)} = 1.$



$$y^{(1)}\left(\boldsymbol{\theta}^{(\mathbf{0})\mathsf{T}}\mathbf{x}^{(1)} + \theta_0^{(0)}\right) = [1 \quad -1]\begin{bmatrix} 1 \\ 3 \end{bmatrix} + 1 = -1 < 0$$

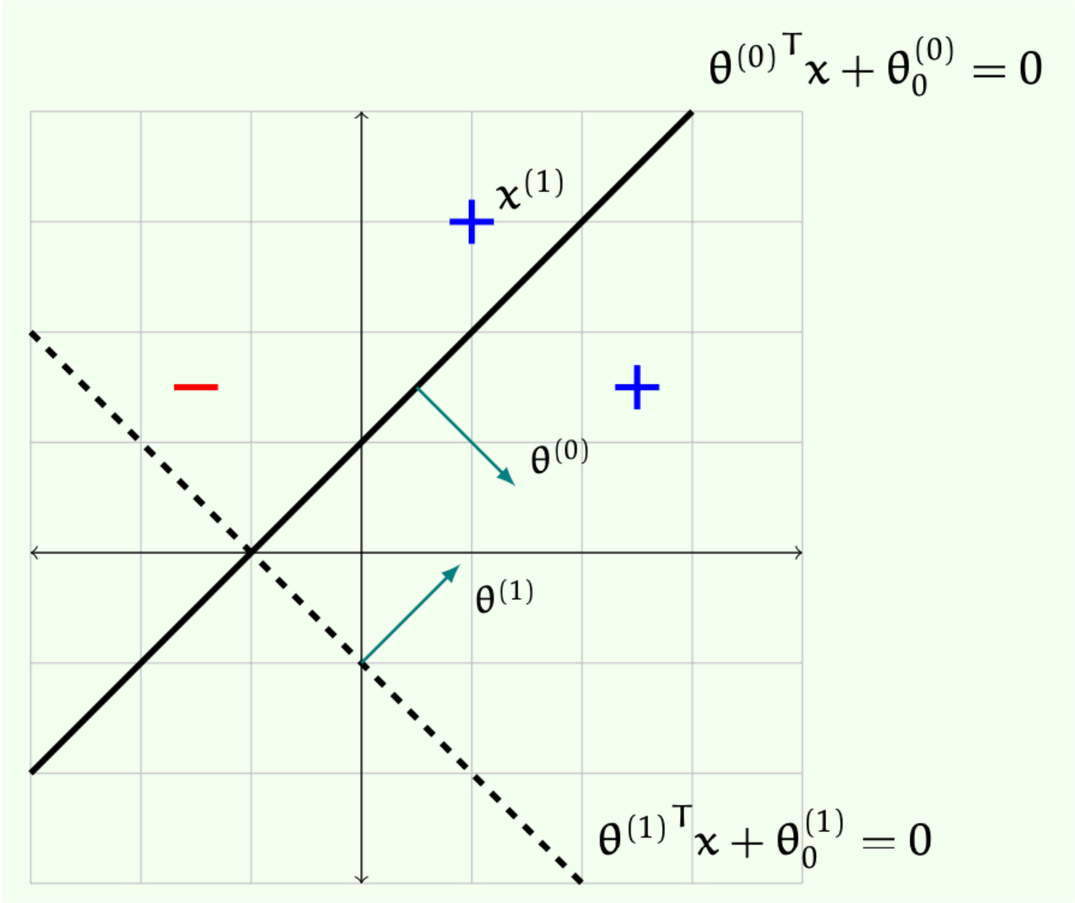
Algorithm

- By running an iteration of the Perceptron algorithm :

$$\theta^{(1)} = \theta^{(0)} + y^{(1)}\mathbf{x}^{(1)} = \begin{bmatrix} 2 \\ 2 \end{bmatrix}$$

$$\theta_0^{(1)} = \theta_0^{(0)} + y^{(1)} = 2$$

- If there is a linear classifier with 0 training error, then this algorithm will eventually find it



Offset

- Sometimes, it can easier to implement or analyze classifiers of the form :

$$h(\mathbf{x};\theta) = \begin{cases} +1, & \text{if } \theta^\top \mathbf{x} > 0 \\ -1, & \text{otherwise} \end{cases}$$

- We can convert any problem involving a linear separator with offset into one with no offset (but of higher dimension).

Offset

- Consider the d-dimensional linear separator defined by $\theta = \begin{bmatrix}\theta_1 & \theta_2 & \cdots & \theta_d\end{bmatrix}$ and offset θ_0 :

$$\mathbf{x}_{\text{new}} = [x_1 \quad x_2 \quad \cdots \quad x_d \quad +1]^\top$$
$$\theta_{\text{new}} = [\theta_1 \quad \theta_2 \quad \cdots \quad \theta_d \quad \theta_0]^\top$$

- Then,

$$\begin{aligned}\theta_{\text{new}}{}^\top \mathbf{x}_{\text{new}} &= \theta_1 x_1 + \cdots + \theta_d x_d + \theta_0 \cdot 1 \\ &= \theta^\top \mathbf{x} + \theta_0\end{aligned}$$

- Thus, θ_{new} is an equivalent ((d + 1)-dimensional) separator to our original, but with no offset.

Offset

- Consider the data set:

$$X = [[1], [2], [3], [4]]$$
$$Y = [[+1], [+1], [-1], [-1]]$$

- It is linearly separable in d = 1 with θ = $[-1]$ and θ_0 = 2.5, but it is not linearly separable through the origin
- Let :

$$X_{\text{new}} = \left[\begin{bmatrix}1\\1\end{bmatrix}\begin{bmatrix}2\\1\end{bmatrix}\begin{bmatrix}3\\1\end{bmatrix}\begin{bmatrix}4\\1\end{bmatrix}\right]$$

- This new dataset is separable through the origin, with $\theta_{\text{new}} = [-1,2.5]^{\top}$.

Offset

Algorithm 4: Perceptron-Through-Origin$(\tau, \mathcal{D}_n)$

```
θ = [0  0  ···  0]^⊤
for t = 1 to n do
    if y^(i)(θ^⊤ x^(i)) ≤ 0 then
        θ = θ + y^(i) x^(i)
return θ
```

Theory of the perceptron

ADVANCED

- **Linear separability:**
  - A training set (D_n) is **linearly separable** if there exist θ, θ_0 such that, for all $i = 1,2,\ldots,n$:

$$y^{(i)}(\theta^\top \mathbf{x}^{(i)} + \theta_0) > 0$$

  - Another way to say:
    - all predictions on the training set are correct:

$$h(\mathbf{x}^{(i)}; \theta, \theta_0) = y^{(i)}$$

    - the training error is zero:

$$\epsilon_n(h) = 0$$

Theory of the perceptron

ADVANCED

- **Convergence theorem**:
  - The basic result about the perceptron is that, if the training data $\mathscr{D}_n$ is linearly separable, then the perceptron algorithm is **guaranteed to find a linear separator**.
    - There are some algorithms that can test for linear separability with run-times $O(n^{\frac{d}{2}})$, $O(d^{2n})$, or $O(n^{d-1} \log n)$
  - Recall that the distance of a point x to the hyperplane $\boldsymbol{\theta}, \theta_0$ is

$$\frac{\theta^\top \mathbf{x} + \theta_0}{||\theta||}$$

  - Then, we'll define the margin of a labeled point (x, y) with respect to hyperplane $\boldsymbol{\theta}, \theta_0$ to be

$$y \cdot \frac{\theta^\top \mathbf{x} + \theta_0}{||\theta||}$$

Theory of the perceptron

ADVANCED

- **Study Question**: What sign does the margin have if the point is incorrectly classified? Be sure you can explain why.

- Now, the **margin of a dataset** (D_n) with respect to the hyperplane θ, θ_0 is the minimum margin of any point with respect to θ, θ_0 :

$$\min_i \left(y^{(i)} \cdot \frac{\theta^\top \mathbf{x}^{(i)} + \theta_0}{\|\theta\|} \right)$$

- The margin is positive if and only if all of the points in the data-set are classified correctly.

Theory of the perceptron

ADVANCED

Perceptron Convergence Theorem.

- *For simplicity, we consider the case where the linear separator must pass through the origin. If the following conditions hold:*
  - (Condition a) there exist θ^* such that $y^{(i)} \frac{\theta^{*\top} \mathbf{x}^{(i)}}{\|\theta^*\|} \geq \gamma$ for $\forall i = 1, \ldots, n$ and for some $\gamma > 0$ and
  - (Condition b) all the examples have bounded magnitude : $\|\mathbf{x}^{(i)}\| \leq R$ for $\forall i = 1, \ldots, n$,
- *Then the perceptron algorithm will make at most* $\left(\frac{R}{\gamma}\right)^2$ *mistakes.* [Proof]

Theory of the perceptron

ADVANCED

- **Proof**:
  - We initialize $\theta^{(0)} = 0$, and let $\theta^{(k)}$ define our hyperplane after the perceptron algorithm has made k mistakes.
  - We are going to think about the angle between the hypothesis we have now, $\theta^{(k)}$ and the assumed good separator θ^*.
  - Since they both go through the origin, if we can show that the angle between them is decreasing usefully on every iteration, then we will get close to that separator.

Theory of the perceptron

ADVANCED

- So, let's think about the cos of the angle between them, and recall, by the definition of dot product:

$$\cos(\theta^{(k)}, \theta^*) = \frac{\theta^{(k)} \cdot \theta^*}{||\theta^*|| \, ||\theta^{(k)}||}$$

- We'll divide this up into two factors,

$$\cos(\theta^{(k)}, \theta^*) = \left(\frac{\theta^{(k)} \cdot \theta^*}{||\theta^*||} \right) \cdot \left(\frac{1}{||\theta^{(k)}||} \right)$$

- And start by focusing on the first factor.

Theory of the perceptron

ADVANCED

- Without loss of generality, assume that the kth mistake occurs on the ith example $(x^{(i)}, y^{(i)})$.

$$\frac{\theta^{(k)}}{\|\theta^*\|} = \frac{(\theta^{(k-1)} + y^{(i)}\mathbf{x}^{(i)}) \cdot \theta^*}{\|\theta^*\|}$$

$$= \frac{\theta^{(k-1)} \cdot \theta^*}{\|\theta^*\|} + \frac{y^{(i)}\mathbf{x}^{(i)} \cdot \theta^*}{\|\theta^*\|}$$

$$\geq \frac{\theta^{(k-1)} \cdot \theta^*}{\|\theta^*\|} + \gamma$$

$$\geq k\gamma$$ (Simple induction - think about it)

- Where we have first applied the margin condition form (Condition a) and then applied simple induction.

Theory of the perceptron

- Now, we'll look at the second factor.

- We note that since $(x^{(i)}, y^{(i)})$ is classified incorrectly, $y^{(i)}\left(\theta^{(k-1)^\top}\mathbf{x}^{(i)}\right) \leq 0.$

$$
\begin{aligned}
\|\theta^{(k)}\|^2 &= \|\theta^{(k-1)} + y^{(i)}\mathbf{x}^{(i)}\|^2 \\
&= \|\theta^{(k-1)}\|^2 + 2y^{(i)}\theta^{(k-1)^\top}\mathbf{x}^{(i)} + \|\mathbf{x}^{(i)}\|^2 \\
&\leq \|\theta^{(k-1)}\|^2 + R^2 \\
&\leq kR^2 \quad \text{(Simple induction - think about it)}
\end{aligned}
$$

- Where we have additionally applied the assumption from (Condition b) and then again used simple induction.

Theory of the perceptron

ADVANCED

- Returning to the definition of the dot product, we have

$$\cos(\theta^{(k)}, \theta^*) = \frac{\theta^{(k)} \cdot \theta^*}{\|\theta^{(k)}\| \|\theta^*\|} = \left(\frac{\theta^{(k)} \cdot \theta^*}{\|\theta^*\|} \right) \cdot \left(\frac{1}{\|\theta^{(k)}\|} \right) \geq (k\gamma) \cdot \frac{1}{\sqrt{k}R} = \sqrt{k} \cdot \frac{\gamma}{R}$$

- Since the value of the cosine is at most 1, we have

$$1 \geq \sqrt{k} \cdot \frac{\gamma}{R}$$

$$k \leq \left(\frac{R}{\gamma} \right)^2$$

Theory of the perceptron

ADVANCED

$$k \leq \left(\frac{R}{\gamma}\right)^2$$

- This result endows the margin γ of $\mathscr{D}_n$ with an operational meaning:
  - When using the Perceptron algorithm for classification, at most $(R/r)^2$ classification errors will be made, where R is an upper bound on the magnitude of the training vectors.

[Recall] Linear classifier

- $h_\theta(x) = \theta^T x$ (in linear regression)
- $h_\theta(x) = \theta^T \phi(x)$ (where $\phi(x)$ is the feature map)

- But real-world input/output relationships are fundamentally non-linear

Non-linear classification

- Supervised learning with non-linear models
  - Training examples: $\{(x^{(i)}, y^{(i)})\}_{i=1}^{n}$
  - Model: $h_\theta(x)$ (non-linear)
  - Goal: learn θ by minimizing a loss function

Non-linear classification

- Regression problems
  - Input : $x^{(i)}$, Output : $y^{(i)} \in \mathbb{R}$
  - Per-example loss:

$$J^{(i)}(\theta) = \frac{1}{2}(h_\theta(x^{(i)}) - y^{(i)})^2$$

  - Total example loss :

$$J(\theta) = \frac{1}{n}\sum_{i=1}^{n} J^{(i)}(\theta) \longrightarrow \text{MSE}$$

  - Model learns by reducing the difference between prediction and target.

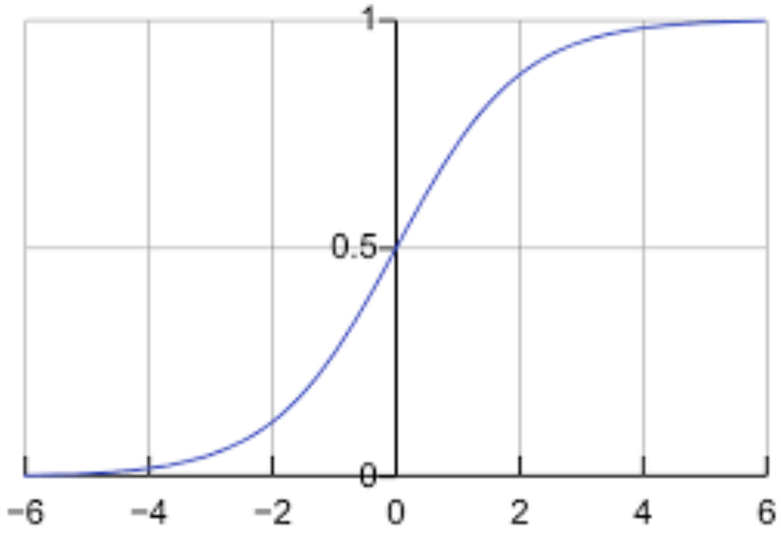
Non-linear classification

- Binary classification
  - Logit : $\bar{h}_\theta(x)$ $(x \in \mathbb{R}^d, \bar{h}_\theta : \mathbb{R}^d \to \mathbb{R})$
  - Probability:

$$h_\theta(x) = \sigma(\bar{h}_\theta(x)) = \frac{1}{1 + e^{-\bar{h}_\theta(x)}} \quad (h_\theta \in [0,1])$$

  - Binary cross-entropy loss:

$$J(\theta) = -\frac{1}{n}\sum_{i=1}^{n}\left[y^{(i)}\log h_\theta(x^{(i)}) + (1 - y^{(i)})\log(1 - h_\theta(x^{(i)}))\right]$$



<sigmoid function>

Non-linear classification

- Multi-class classification
  - Logit : $\bar{h}_\theta(x)$ ($x \in \mathbb{R}^d$, $\bar{h}_\theta : \mathbb{R}^d \to \mathbb{R}^k$)
  - Each logit corresponds to the prediction for one of the k classes.

  - Softmax:

$$\hat{y}_c^{(i)} = p(y = c \mid x^{(i)}; \theta) = \frac{e^{\bar{h}_{\theta,c}(x^{(i)})}}{\sum_{s=1}^{k} e^{\bar{h}_{\theta,s}(x^{(i)})}}$$

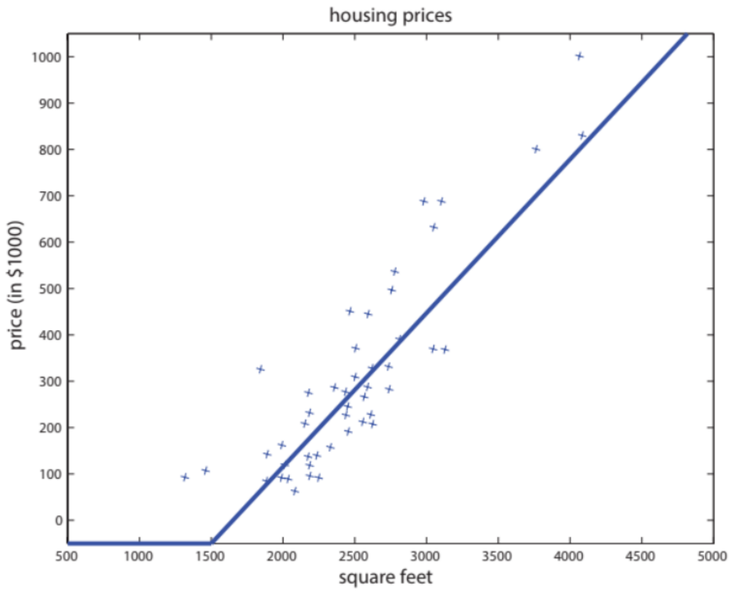
  - Cross-entropy loss

$$J(\theta) = -\frac{1}{n} \sum_{i=1}^{n} \sum_{c=1}^{k} y_c^{(i)} \log \hat{y}_c^{(i)}$$

Neural networks

- What is a neural networks?
  - Built from repeated matrix multiplication and non-linear activation.

- Let's think housing price prediction example.
  - Given the size of the house, we want to predict the price.
  - Prevent negative housing price.



Neural networks

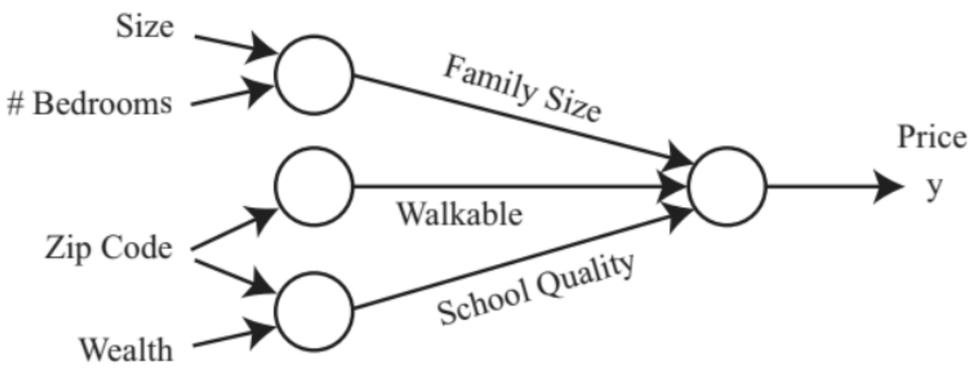
- $\bar{h}_\theta : x \to y$
- $\bar{h}_\theta(x) = \max(wx + b, 0)$, where $\theta = (w, b) \in \mathbb{R}^2$
- The function $max\{t, 0\}$ is called a ReLU.

- ReLU is often referred to as an activation function.
- $\bar{h}_\theta(x) = \text{ReLU}(w^T x + b)$, where $w \in \mathbb{R}^d, b \in \mathbb{R}$, and $\theta = (w, b)$
  - Neural network with a single neuron
  - w: weight vector, b : bias

Neural networks

- Stacking Neurons

- Housing prediction example.
  - Inputs: size, # bedrooms, zip code, neighborhood wealth
  - Size, # bedrooms → family size
  - Zip code → walkable, school quality
  - Neighborhood wealth → school quality

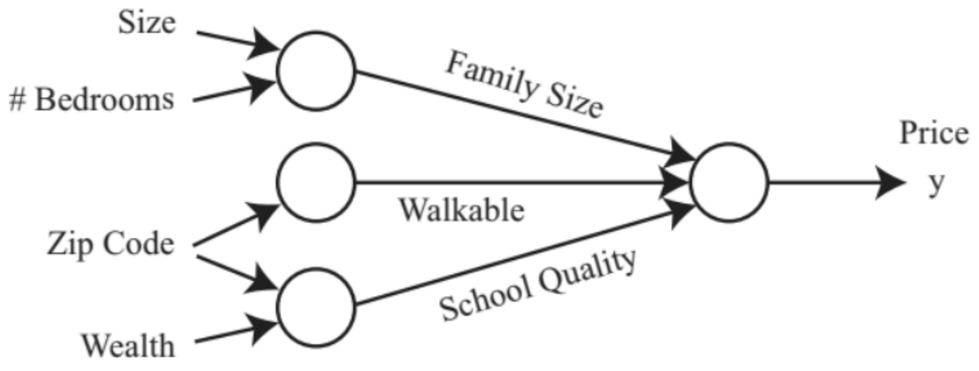


Neural networks

- "Family size", "walkable", "school quality" $\rightarrow a_1, a_2, a_3$
  - a_i's often referred to as "hidden units" or "hidden neurons"

$$a_1 = \mathrm{ReLU}(\theta_1 x_1 + \theta_2 x_2 + \theta_3)$$
$$a_2 = \mathrm{ReLU}(\theta_4 x_3 + \theta_5)$$
$$a_3 = \mathrm{ReLU}(\theta_6 x_3 + \theta_7 x_4 + \theta_8)$$

$$\bar{h}_\theta(x) = \theta_9 a_1 + \theta_{10} a_2 + \theta_{11} a_3 + \theta_{12}$$



Neural networks

- Two-layer Fully Connected Neural Networks

- For general case
  - Calculate intermediate variable as a function of all x_i

$$
\begin{aligned}
\forall j \in \{1,\ldots,m\}, \quad z_j &= w_j^{[1]T} x + b_j^{[1]}, \quad \text{where } w_j^{[1]} \in \mathbb{R}^d,\ b_j^{[1]} \in \mathbb{R} \\
a_j &= \mathrm{ReLU}(z_j), \\
a &= [a_1, \ldots, a_m]^T \in \mathbb{R}^m \\
\bar{h}_\theta(x) &= w^{[2]T} a + b^{[2]}, \quad \text{where } w^{[2]} \in \mathbb{R}^m,\ b^{[2]} \in \mathbb{R}\,.
\end{aligned}
$$

Neural networks

- Vectorization
- Before vectorization we should calculate one-by-one using for-loop

$$
\begin{aligned}
z_1 &= w_1^{[1]T} x + b_1^{[1]} \\
z_2 &= w_2^{[1]T} x + b_2^{[1]} \\
&\vdots \\
z_m &= w_m^{[1]T} x + b_m^{[1]}
\end{aligned}
$$

- After vectorization

$$
z = \begin{bmatrix} z_1 \\ z_2 \\ \vdots \\ z_m \end{bmatrix} \in \mathbb{R}^m \,,\, b^{[1]} = \begin{bmatrix} b_1^{[1]} \\ b_2^{[1]} \\ \vdots \\ b_m^{[1]} \end{bmatrix} \in \mathbb{R}^m \longrightarrow z = W^{[1]} x + b^{[1]} \,,\, a = \begin{bmatrix} \mathrm{ReLU}(z_1) \\ \mathrm{ReLU}(z_2) \\ \vdots \\ \mathrm{ReLU}(z_m) \end{bmatrix}
$$

- 2-layer fully-connected network

$$
\begin{aligned}
a &= \mathrm{ReLU}(W^{[1]} x + b^{[1]}) \\
\bar{h}_\theta(x) &= W^{[2]} a + b^{[2]}
\end{aligned}
$$

Neural networks

- Multi-layer fully-connected neural networks (using vectorized notation)
- We can stack more layers to get a deeper fully-connected nn.

$$
\begin{aligned}
a^{[1]} &= \mathrm{ReLU}\left(W^{[1]}x + b^{[1]}\right) \\
a^{[2]} &= \mathrm{ReLU}\left(W^{[2]}a^{[1]} + b^{[2]}\right) \\
&\vdots \\
a^{[r-1]} &= \mathrm{ReLU}\left(W^{[r-1]}a^{[r-2]} + b^{[r-1]}\right) \\
\bar{h}_\theta(x) &= W^{[r]}a^{[r-1]} + b^{[r]}
\end{aligned}
$$

Neural networks

- Other activation functions

$$\sigma(z) = \frac{1}{1+e^{-z}} \quad \text{(sigmoid)}$$

$$\sigma(z) = \frac{e^{z}-e^{-z}}{e^{z}+e^{-z}} \quad \text{(tanh)}$$

$$\sigma(z) = \max\{z, \gamma z\}, \quad \gamma \in (0,1) \quad \text{(leaky ReLU)}$$

- $$\sigma(z) = \frac{z}{2}\left[1 + \operatorname{erf}\left(\frac{z}{\sqrt{2}}\right)\right] \quad \text{(GELU)}$$

$$\sigma(z) = \frac{1}{\beta}\log(1+\exp(\beta z)), \quad \beta > 0 \quad \text{(Softplus)}$$

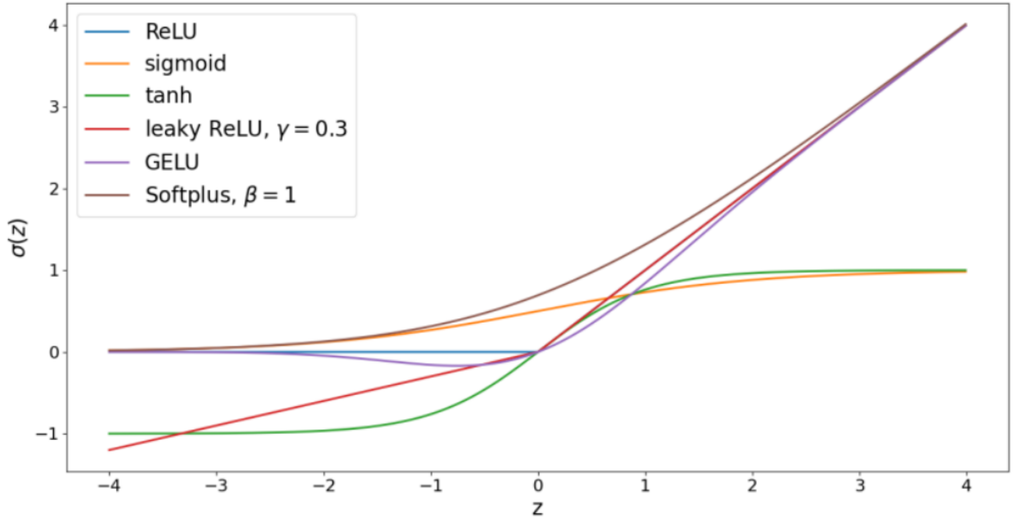


Figure 7.3: Activation functions in deep learning.

- **Study Question:** Why do we not use the identity function?

Neural networks

- Connection to the Kernel Method
- Recall: $\theta^T \phi(x)$ (θ: parameters, $\phi(x)$: feature map, which is a handcrafted function)

- We can view deep learning as a way to automatically learn the right feature map.

$$\bar{h}_\theta(x) = W^{[r]}\phi_\beta(x) + b^{[r]}\,, (\beta : \;\; a^{[r-1]} = \phi_\beta(x))$$

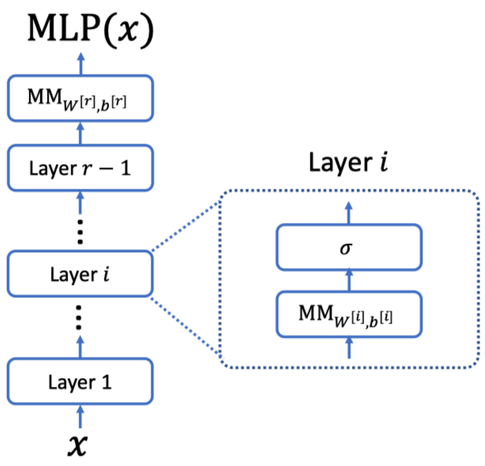
- When β is fixed, then $\phi_\beta(\,\cdot\,)$ can viewed as a feature map, $\bar{h}_\theta(x)$ is just a linear model over the features.

- However we will train both β and the parameters $W^{[r]}, b^{[r]}$ are optimized, and therefore we are not learning a linear model but also a good feature map $\phi_\beta(\,\cdot\,)$

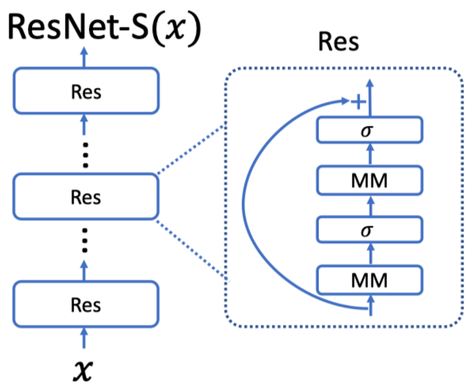
Modern Neural networks

- Residual Network (ResNet)
- Convolution Network (CNN)
- Attention (Transformer)
- LayerNorm

Modern Neural networks: Residual Network



Normal Layers



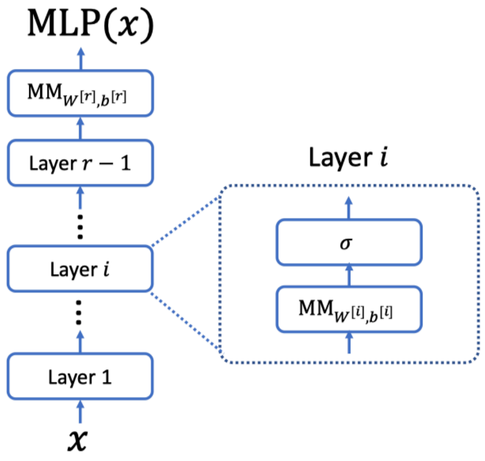
w/ Residual Connection

Residual network:

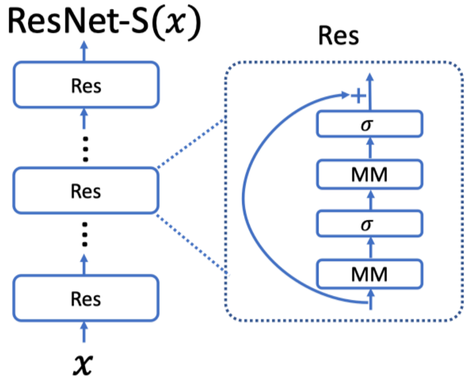
$$Res(z) = z + \sigma(MM(\sigma(MM(z)))) \,.$$

- One of the most influential network designs in vision
- Widely used in modern large-scale architectures
- Residual connection

Modern Neural networks: Residual Network



Normal Layers



w/ Residual Connection

Residual Layers vs. Normal Layers

$$ResNet\text{-}S(x) = MM(Res(Res(\cdots Res(x)))) \,.$$

- Use skip connections
- Learn residual function
- Combine with identity shortcut
- Simplified ResNet: composition of multiple residual blocks

Modern Neural networks: ConvNet

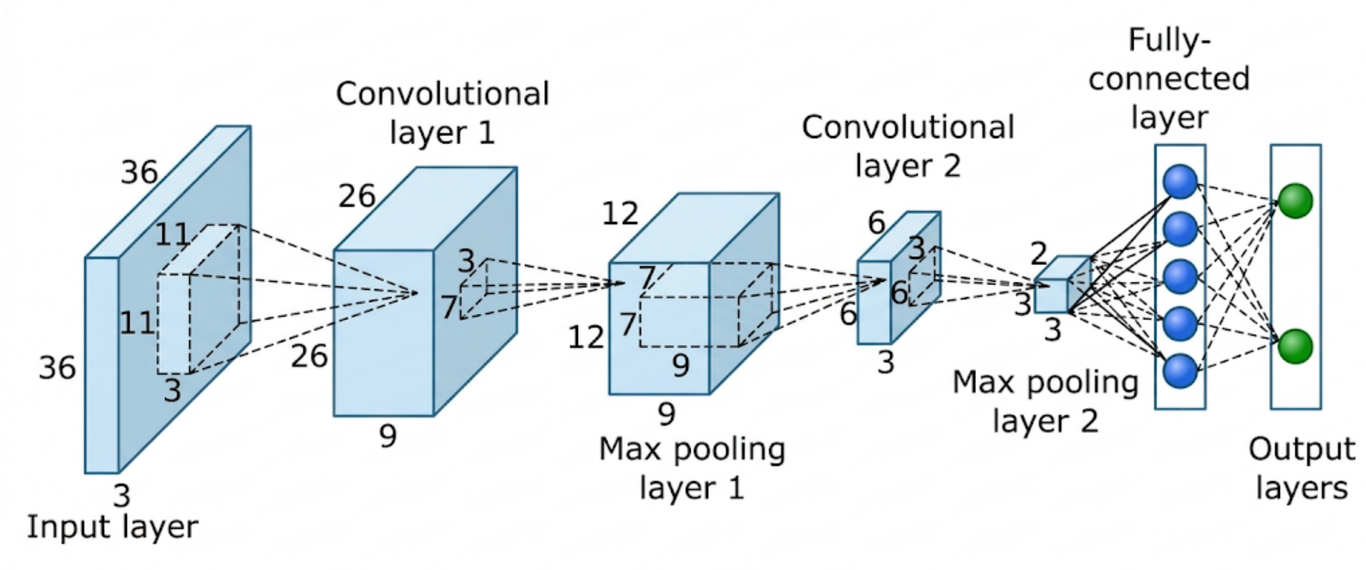


Fig. A hierarchical 2D convolutional neural network pipeline

- Automatically learns complex spatial features via $k \times k$ filters and multi-channel 3D tensors
- Designed for computer vision applications that processes 2D inputs
- Utilizes specialized modules like **max-pooling** for effective downsampling before final classification.

Modern Neural networks: Transformer

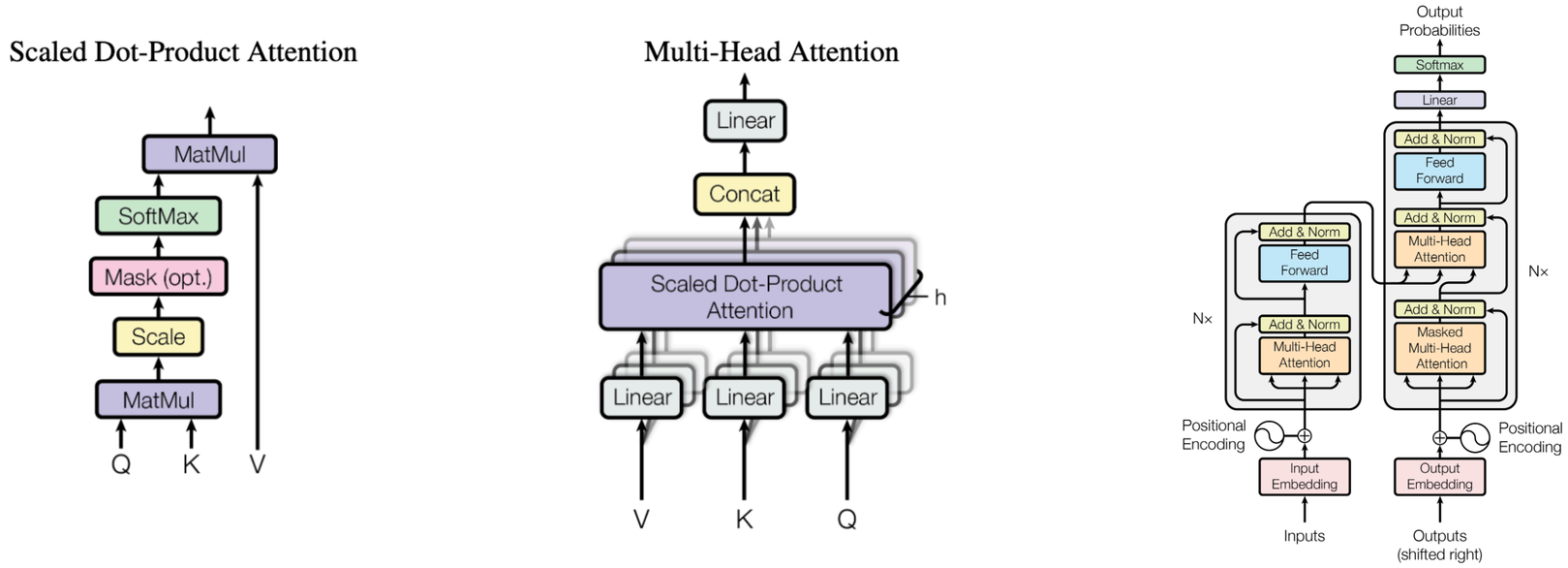


Fig. Self-attention and transformer architecture

- Self-attention mechanism models global dependencies across a sequence by computing relationships between all tokens.
- Processes entire sequences in **parallel** using Query (Q), Key (K), and Value (V) operations.
- Forms the structural foundation for modern **Large Language Models (LLMs)**.

Modern Neural networks: Vision Transformer

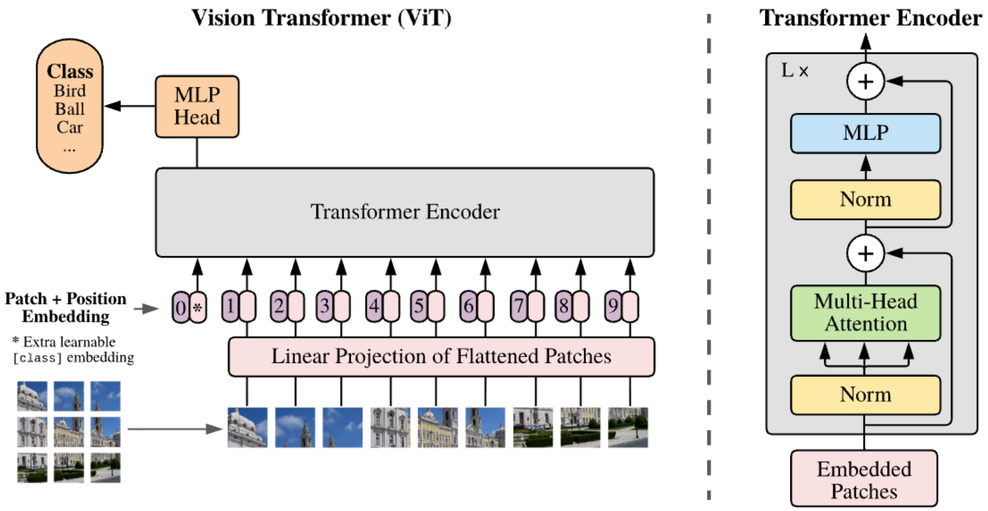
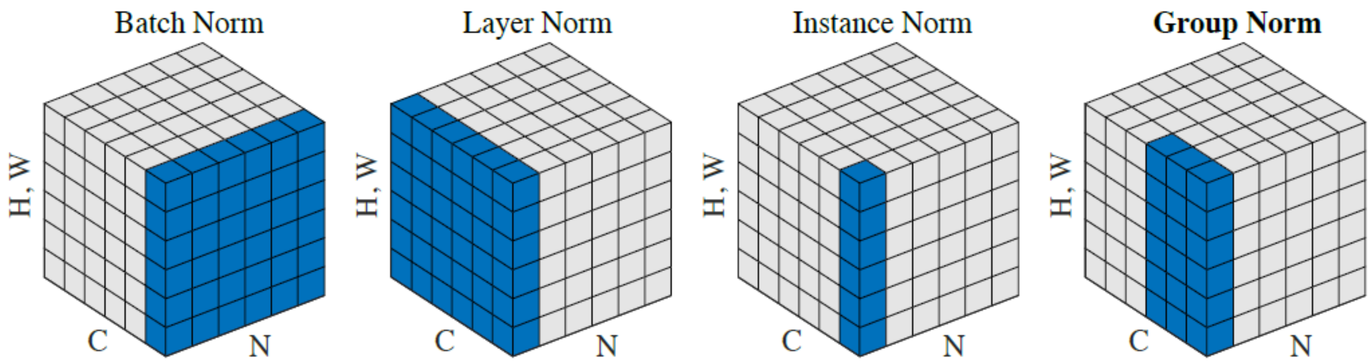


Fig. Vision transformer

- Adapts Transformer to vision by partitioning an image into a sequence of flattened **patches.**
- Captures **global spatial context** from initial layers,
  bypassing the local receptive field limits of CNNs.
- Achieves excellent performance through **large-scale pre-training** on massive datasets.

Modern Neural networks: LayerNorm



Layer normalization

- Normalize hidden features for each sample
- Mean and variance computed within the sample
- Independent of mini-batch statistics

$$\mathrm{LN}(x) = \gamma \odot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

QnA

- Any questions?
- Note: this lecture notes are largely adopted from
  - MIT 6.036(2019 Fall), Stanford CS229(2023-2024 Summer)